
INTERNSHIP TRAINING REPORT

on

FULLSTACK TECHNOLOGY

(React.js, Fast API, SQL)

Submitted in partial fulfilment of the requirements
for the award of degree

BACHELOR OF COMPUTER APPLICATIONS

of

KLE TECHNOLOGICAL UNIVERSITY

by

MS. SARVAMANGALA M C

USN: 01FE23BCA205

DEPARTMENT OF COMPUTER APPLICATIONS

KLE TECHNOLOGICAL UNIVERSITY

Vidyanagar, Hubballi-580031 Karnataka.

May- 2026

Internship Training Report

on

FULLSTACK TECHNOLOGY

(React.js, Fast API, SQL)

Submitted in partial fulfilment of the requirements for
the award of degree

BACHELOR OF COMPUTER APPLICATIONS

of

KLE TECHNOLOGICAL UNIVERSITY

by

MS. SARVAMANGALA M C

USN: 01FE23BCA205

under the guidance of

Internal Guide:

Smt. Shashikala Budni
Assistant Professor,
Dept. of Computer Applications

External Guide:

Ms. Kasturi Murthy
Mr. Nandeesh Kodipalli
Ms. Vidya Patil,
ION IDEA, Bengaluru

DEPARTMENT OF COMPUTER APPLICATIONS

KLE TECHNOLOGICAL UNIVERSITY

Vidyanagar, Hubballi-580031 Karnataka.

May- 2026

DEPARTMENT OF COMPUTER APPLICATIONS
KLE TECHNOLOGICAL UNIVERSITY

CERTIFICATE

This is to certify that the internship training work entitled

" FULLSTACK TECHNOLOGY"

(React.js, Fast API, SQL)

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Computer Applications of

KLE Technological University, Hubballi, Karnataka

is a result of the bonafide work carried out by

MS. SARVAMANGALA M C

USN: 01FE23BCA205

during the academic year 2025-2026

Smt. Shashikala Budni

Project Guide ,

Assistant Professor,

Dept. of Computer Applications

Dr. P. R. Patil ME, PhD

Head of Department ,

Professor,

Dept. of Computer Applications

Prof. B.S. Anami

Registrar,

KLE Technological

University

Viva-Voce Examination

Name of the Examiners

Signature with Date

1. _____

: _____

2. _____

: _____

ACKNOWLEDGEMENT

Every successful completion of any undertaking would be complete only after we remember and thank the Almighty, the parents, the teachers, and the personalities who directly or indirectly helped and guided during the execution of that work. The success of this work is equally attributed to all the well-wishers who have encouraged and guided throughout the execution.

I express my deepest gratitude to **Smt. Shashikala Budni**, Internal Guide, and **Ms. Kasturi Murthy, Mr. Nandeesh Kodipalli, and Ms. Vidya Patil, and of ION IDEA, Bengaluru**, External Guides, for their continuous guidance and assistance throughout the project with great interest.

I avail this opportunity to express my deepest gratitude to the Head of Department for encouraging me with valuable tips and insights throughout the duration of this project.

I am grateful to **Prof. B.S. Anami, Registrar**, for his blessings. I am grateful to KLE Technological University, Hubballi, for providing the opportunity to undertake this internship and work on a meaningful real-world project that has contributed to my academic and professional development.

I also thank all the teaching and non-teaching staff members of the BCA department for their invaluable cooperation and support throughout the course of this internship.

Finally, I would like to express my sincere thanks to my Parents and Friends for their enormous encouragement and to all others who have extended their helping hands towards the successful completion of this project.

Sarvamangala M C

ORGANIZATION PROFILE

Company Name: ION IDEA

Website: www.ionidea.com

Industry: Information Technology & Software Development

Location: Bengaluru, Karnataka, India

Company Type: Private Limited

About the Organization

ION IDEA is a Bengaluru-based software product and services company specializing in enterprise application development, digital transformation, and education technology solutions. The company leverages modern web technologies and agile development practices to deliver high-quality software products for clients across various industries including education, healthcare, and enterprise services. With a strong focus on innovation and quality, ION IDEA builds scalable and maintainable systems that drive measurable business impact.

Vision

To be a leading technology company that empowers organizations through innovative, scalable, and reliable software solutions that drive digital transformation and sustainable growth.

Mission

To deliver world-class software solutions using cutting-edge technologies, with a commitment to quality, customer satisfaction, and continuous innovation.

Core Service Areas

- Enterprise Web Application Development
 - Education Technology (EdTech) Solutions
 - Backend API Development and Integration
 - Cloud Deployment and DevOps
 - Digital Transformation Consulting
-

ABSTRACT

The Internship Training Report on “Full Stack Technology on (React.js, Fast API, SQL)” was undertaken during an internship at ION IDEA, Bengaluru. The frontend development, focused on building a React-based user interface for a comprehensive Course Learning and Attendance Management System. The project utilized modern frontend technologies including React.js, TypeScript, and Tailwind CSS to create responsive, type-safe components for modules such as user authentication, course and curriculum management, CO-PO mapping, assessment entry, attainment calculation, and report generation. The frontend architecture implemented component-based design patterns, state management with React hooks and Context API, RESTful API integration with the Fast API backend, and performance optimizations including code splitting and memoization. Key features included role-based access control, real-time data synchronization, responsive design for cross-device compatibility, and comprehensive error handling and user feedback mechanisms.

The internship provided extensive hands-on experience with professional development workflows including Git-based version control, agile sprint planning, and collaborative coding practices. Technical skills acquired included mastery of React.js component lifecycle, TypeScript type safety, state management strategies, API integration patterns, and responsive design principles. The project successfully delivered a scalable, institution-ready Outcome-Based Education (OBE) management platform that streamlines educational administration processes, improves data accuracy through automated calculations, and supports NBA/NAAC accreditation compliance through comprehensive reporting capabilities. The experience established a strong foundation in modern frontend development practices and full-stack web application architecture.

TABLE OF CONTENTS

SI.NO	CONTENTS	Pg No
1	Introduction	1
2	Organization Profile	3
3	Objectives of the Internship	5
4	Overview of React.js and Frontend Development	7
5	Tasks and Projects Undertaken	11
6	Skills Acquired	14
7	Challenges Faced	16
8	Conclusion	21
9	Appendix	23
9.1	Internship Certificate	23
9.2	Screenshots of training	24
9.3	References	27

CHAPTER 1

INTRODUCTION

Modern engineering education requires specialized digital systems to efficiently manage student attendance, course delivery, and academic scheduling. This project focuses on designing and developing a dedicated Course Learning and Attendance Management System. By utilizing Fast API for the backend, React for the frontend, and MariaDB or SQL for robust data storage, the system provides a high-performance solution for tracking student engagement and learning milestones.

Internship Overview

As part of the academic curriculum for the Bachelor of Computer Applications programme at KLE Technological University, I undertook an internship training on “**Full Stack Technology (React.js, Fast API, SQL)**” at ION IDEA, Bengaluru. This internship was conducted during the final semester and focused on building a production-grade Frontend system using React.js to support a comprehensive OBE management platform for academic institutions.

The primary objective of the internship was to design and develop RESTful API endpoints that support core academic workflows such as course management, CO-PO mapping, marks entry, attainment computation, and report generation. The project addressed a genuine institutional need and provided the intern with hands-on exposure to real-world software development practices.

Duration, Organization, and Technology Stack

The internship was carried out over a period of approximately 12 weeks, from February 20, 2025 to May 15, 2025, under the mentorship Ms. Kasturi Murthy, Mr. Nandeesh Kodipalli, and Ms. Vidya Patil at ION IDEA, Bengaluru. The following technologies and tools were employed during the project:

- Programming Languages: Python, JavaScript
- Backend Framework: Fast API (Python)

- Frontend Framework: React
- Database: PostgreSQL / SQLite (via SQL Alchemy ORM)
- API Testing: Postman, Swagger UI (auto-generated by FastAPI)
- Version Control: Git and GitHub
- Other Tools: Unicorn (ASGI server), Pydantic (data validation), Alembic (migrations)

Purpose and Learning Expectations

The internship aimed to bridge the gap between theoretical learning and practical implementation by involving real-world problem-solving in the education technology domain. Through this experience, the following outcomes were expected:

- Design and implement scalable RESTful APIs using the React.js framework.
- Gain hands-on experience in frontend architecture, database design, and data modelling.
- Understand how OBE(outcome based education) frameworks work and how they can be digitized effectively.
- Enhance collaborative development skills through team-based agile workflows.

This report documents the methodologies, design principles, technical challenges, and outcomes encountered throughout the internship, highlighting the application of modern frontend development techniques to solve education administration challenges.

CHAPTER 2

COMPANY PROFILE: ION IDEA, BENGALURE

Company Overview

Name: ION IDEA

Location: Bengaluru, Karnataka, India

Website: www.ionidea.com

Industry: Information Technology & EdTech

Year of Establishment and Domain Expertise

ION IDEA is a software technology firm based in Bengaluru, India, with expertise in delivering enterprise-grade web applications and education technology platforms. The company has worked extensively on outcome-based education systems, accreditation support tools, and academic data management solutions for engineering colleges and universities across India. Their domain expertise encompasses:

- Outcome-Based Education (OBE) Platform Development
- Enterprise Resource Planning (ERP) for Academic Institutions
- API Development and Microservices Architecture
- Frontend Development with React and Angular
- Database Design, Optimization, and Migration

Team Structure and Assigned Department

The core leadership team at ION IDEA includes experienced software engineers and product managers who bring expertise across full-stack development, cloud infrastructure, and EdTech product design. The external guides for this internship were:

- Ms. Kasturi Murthy – Software Architect
- Mr. Nandeesh Kodipalli – Senior Backend Developer
- Ms. Vidya Patil – Product Manager / Technical Lead

As an intern, I was assigned to the Frontend Development team, specifically contributing to the Management System project. My responsibilities included UI for frontend designing, database schemas, and integrating frontend logic with the Fast API backend developed by the senior team.

Company's Role in the EdTech Industry

ION IDEA plays a significant role in the education technology ecosystem in India by providing institutions with the digital tools needed to comply with accreditation standards such as NBA (National Board of Accreditation) and NAAC. Their OBE platform enables institutions to automate the computation of course outcomes, program outcomes, and attainment levels—tasks that are otherwise extremely time-consuming when done manually. Through its work, ION IDEA contributes meaningfully to the modernization of academic administration in India.

CHAPTER 3

OBJECTIVES OF THE INTERNSHIP

- To design a structured database schema in MariaDB for managing students, courses, and daily attendance records.
- To develop RESTful APIs using Fast API for high-performance data processing.
- To build responsive frontend interfaces using React for faculty to mark attendance and for students to track progress.
- To implement secure data transactions and ensure ACID compliance for critical attendance records.
- To follow clean and maintainable coding practices for the integration of all system layers.

1. Understand Real-World Software Development Workflow

The project focused on mastering the professional software development lifecycle by transitioning institutional data from static Excel sheets into a dynamic React and Fast API ecosystem. Throughout the workflow, I executed critical stages including requirement analysis for the attendance logic, relational database schema design for MySQL tables, and interactive API documentation using Swagger. By adopting an Agile methodology and utilizing Git for version control, I learned to manage complex frontend states—such as the Draft/Finalized workflow—and implement frontend conflict-detection for scheduling. This comprehensive approach provided deep insights into building scalable, industry-standard systems where code maintainability, API-driven architecture, and rigorous testing with Postman are essential for transforming raw data into actionable academic insights.

2. Gain Hands-On Experience with React and Fast API Technologies

The primary objective was to master React.js by transforming institutional data into a dynamic, state-driven user interface. I gained hands-on experience in building a component-based architecture, creating reusable UI elements for the attendance and marks modules to ensure code maintainability. By implementing advanced state management, I successfully handled complex frontend logic such as the Draft/Finalized record states and real-time notification tracking. Furthermore, I focused on seamless API integration using

asynchronous data fetching to connect with the Fast API backend, ensuring that data-heavy reports from the MySQL tables render efficiently across a fully responsive UI optimized for both desktop and mobile devices.

3. Work on Real-Time Projects with Team Collaboration

The internship provided the opportunity to work on a live, production-bound project that directly addresses challenges faced by academic institutions undergoing OBE compliance. I collaborated within a development team, contributing to sprint planning, code reviews, technical discussions, and integration testing. Through regular feedback from the external guides, I developed the ability to write clean, documented, and testable code—an essential skill for any software engineer.

4. Learn API Design, Testing, Security, and Deployment Practices

The final objective focused on mastering the design, testing, and deployment of APIs to ensure a seamless connection between the React frontend and the FastAPI backend. I gained hands-on experience in API Design by structuring standardized RESTful endpoints to manage academic data across the relational tables. For Testing and Documentation, I utilized Postman for structured endpoint validation and leveraged Swagger UI for real-time interactive testing and schema verification. I also learned essential Deployment Practices by configuring the application on a development server using Uvicorn, while managing environment variables to ensure a scalable and maintainable system. These practices provided a comprehensive understanding of how to build and maintain secure, production-grade communication layers between frontend and backend services.

CHAPTER 4

OVERVIEW OF REACT.JS AND FRONTEND DEVELOPMENT

React.js is the core library used to build the interactive user interface, allowing the system to handle complex data updates without page refreshes.

1. Key Features of React.js

- **Component-Based Architecture:** The UI is split into small, reusable pieces (e.g., Attendance Buttons, Marks Tables, Timetable Grids), ensuring a clean and maintainable codebase.
- **Virtual DOM:** Only the specific parts of the page that change are re-rendered, which is critical for displaying heavy datasets like the "Consolidated Student Marks Report."
- **State Management (Hooks):** Uses `useState` and `useEffect` to manage dynamic data flows, such as the transition between Draft and Finalized status.
- **Declarative UI:** Makes the code more predictable and easier to debug by describing how the UI should look for every specific state (e.g., Loading vs. Success).
- **Axios/Fetch Integration:** Seamlessly communicates with Fast API using asynchronous calls to fetch and post data in JSON format.

2. RESTful API Design Principles (Frontend Perspective)

In this project, the communication between the React.js frontend and the backend follows the REST (Representational State Transfer) architectural style. This ensures that the frontend can interact with data in a standardized, predictable way.

1. Resource-Oriented Architecture

The API is organized around specific "Resources" (the nouns of the system). Instead of complex commands, the frontend makes requests to clean, intuitive URLs that represent the data it needs:

- `/students` (To manage student profiles)
- `/courses` (To access subject details)

- /attendance (To handle the tracking)
- /marks (To fetch or update academic results)
- /reports (To generate consolidated summaries)

2. Standard HTTP Methods

The React frontend uses specific HTTP to tell the backend exactly what action to perform on a resource:

- GET: Used to retrieve data. (e.g., Fetching the timetable grid or a student's marks).
- POST: Used to create new data. (e.g., Adding a new student registration or submitting a finalized attendance sheet).
- PUT/PATCH: Used to update existing records. (e.g., Editing a subject's details or updating a "Seen-Flag" notification).
- DELETE: Used to remove data. (e.g., Deleting an incorrect entry from the course list)

3. Database Design and ORM with SQL Alchemy

The project used SQL Alchemy as the Object Relational Mapper (ORM) to interact with the relational database. This approach abstracted raw SQL queries into Python class definitions (models), making the code more maintainable and portable. Key database entities designed during the internship included:

- User (admin, faculty, student roles)
- Department and Programme
- Course and Syllabus
- CourseOutcome (CO) and ProgramOutcome (PO)
- CO-PO Mapping (with correlation levels)
- Assessment Component and Marks
- Attainment Record

Alembic was used for database migrations, allowing schema changes to be versioned and applied systematically without data loss.

4 Authentication and Security

The frontend ensures data protection through a secure, role-based verification system.

- **Secure Token Flow:** After login, the React app stores a unique authentication token and attaches it to the header of every API request to verify identity.
- **Role-Based Access (RBAC):** The UI dynamically changes based on the user. Students are blocked from faculty-only pages like "Marks Entry," while Admins are the only ones who can see "User Management."
- **Protected Routes:** React prevents unauthorized users from accessing internal dashboards by automatically redirecting them to the login page if a valid token is missing.

5. React Libraries and Tools Used

The implementation of the frontend was supported by a rich ecosystem of JavaScript libraries and tools:

- **React 18 & TypeScript:** The core foundation that ensures the website is fast (React) and the code is free of errors through strict data rules (TypeScript).
- **Axios:** The "messenger" that fetches your attendance and marks data from the Fast API backend.
- **React Router:** The "traffic controller" that moves users between pages and prevents students from accessing faculty-only areas.
- **Tailwind CSS:** The design tool that ensures the dashboard looks professional and works perfectly on mobile phones and desktops.
- **PDF/XLSX:** The export tools that allow users to click a button and instantly download their reports as PDF or Excel files.
- **NPM (Node Package Manager):** Used for managing all frontend dependencies and ensuring version consistency across the development team.

6. Integration with React Frontend

- **Client-Server Model:** Operates on a decoupled architecture where React (Port 3000) handles the UI and FastAPI (Port 8000) manages the business logic.
- **Axios Interceptors:** Uses a centralized Axios instance to automatically inject JWT Bearer Tokens into headers for secure, authenticated API requests.
- **Service Layer Pattern:** Implements dedicated API files (e.g., attendanceApi.ts) to abstract HTTP logic, keeping components clean and focused on the UI.

- **State Synchronization:** Uses React Hooks to fetch JSON data and update the component state, ensuring the dashboard reflects real-time database changes.
- **Environment Control:** Manages API endpoints through .env files, allowing seamless switching between development and production environments.
- **CORS & Security:** Backend is configured with Cross-Origin Resource Sharing to safely authorize requests from the React frontend.
- **Error Handling:** Standardized try-catch blocks paired with toast notifications provide instant user feedback for failed network or validation requests.

CHAPTER 5

TASKS AND PROJECTS UNDERTAKEN

During the internship at ION IDEA, Bengaluru, I worked on the " **Design and Development of course learning and attendance Management System** " project. My contributions were primarily focused on frontend development using React.js, covering UI testing, API design, database modelling, business logic implementation, and integration testing.

1. Frontend Tasks:

- **Authentication & Role Security:** Developed a secure login system featuring JWT token management via Axios interceptors and implemented Role-Based Access Control (RBAC) to restrict page access for Admins, Faculty, and Students.
- **Dynamic Layout & Navigation:** Designed a fully responsive vertical dashboard with a mobile-optimized sidebar and dynamic routing to ensure a seamless user experience across all devices.
- **Cascading Academic Management:** Built intelligent dropdown systems for Curriculum, Term, and Course management that use cascading logic to fetch data based on previous user selections.
- **Attendance Marking System:** Engineered a high-performance attendance interface with student list toggles, session timing selection, and a batch-saving feature for efficient daily record management.
- **Student Analytics View:** Created a comprehensive student attendance portal featuring summary tables, day-wise detailed views, and automated percentage calculations.
- **Automated Reporting & Exports:** Implemented client-side generation logic using **jsPDF** and **XLSX** to allow users to export attendance and attainment reports directly as PDF or Excel files.
- **Form Validation & Type Safety:** Utilized **React Hook Form** combined with **Zod/Yup** schemas and **TypeScript** interfaces to ensure all user inputs are validated and type-safe before reaching the backend.

- **Timetable & Topic Tracking:** Developed a responsive grid-based timetable interface and a topic management module to track instructor assignments and real-time schedule updates.
- **Global State & API Integration:** Architected a modular API service layer using custom React Hooks and Axios to manage server state and handle global environment configurations.
- **Performance & Error Handling:** Optimized application speed through code splitting (React.lazy) and implemented a robust error-handling system with Toast notifications and Error Boundaries.

2. Database Schema Design

A significant portion of the internship involved designing and refining the relational database schema. This included:

- Defining normalized table structures for all entities using SQLAlchemy models
- Establishing foreign key relationships between courses, outcomes, and assessment records
- Writing Alembic migration scripts to apply schema changes incrementally
- Optimizing queries for attainment computation, which involved joining multiple tables

3. API Testing and Documentation

All developed API endpoints were rigorously tested using Postman. Test collections were organized by module and included tests for valid inputs, edge cases, and error handling scenarios. FastAPI's built-in Swagger UI was used to share live API documentation with the frontend team, enabling seamless integration. Key testing tasks included:

- Writing Postman test scripts for all CRUD endpoints
- Validating JWT authentication flows including token expiry handling
- Testing CO attainment computation accuracy against manually calculated reference values

4. Deployment Steps

The frontend code was maintained in a shared GitHub repository. I worked on feature branches and raised pull requests for review by the senior developers. This exposed me to:

- Git branch management (feature branches, merging, conflict resolution)
- Code review feedback and iterative improvement
- Integrating backend endpoints with the React frontend for end-to-end testing

5. Tools and Technologies Used

- React: For building the dynamic user interface.
- FastAPI: For high-performance backend API services.
- MariaDB: For secure and scalable relational data storage.
- REST APIs: For seamless communication between system components.
- Git & VS Code: For version control and efficient development.

CHAPTER 6

SKILLS ACQUIRED

The internship at ION IDEA provided a comprehensive learning experience, allowing me to strengthen my technical foundation, explore professional tools, and develop essential soft skills. This section summarizes the key skills acquired during the internship.

1. Technical Skills

- Core Framework: UI built with React 18 and TypeScript for type-safe components.
- API & Security: Secure Axios setup with JWT for authenticated data flow.
- Navigation: Role-based access control via React Router 6 protected routes.
- UI/UX Design: Responsive, mobile-first layouts using Tailwind CSS and Bootstrap.
- Data Grids: High-performance records management using AG Grid and TanStack.
- Analytics: Interactive attainment charts developed with Recharts.
- Validation: Schema-based form handling using React Hook Form and Zod.
- Reporting: Automated PDF and Excel exports via jsPDF and XLSX.
- Optimization: Faster load times through code splitting and memoization.

2. Tools and Software Used

- React: For building the dynamic user interface for frontend.
- FastAPI: For high-performance backend API services for backend.
- MariaDB: For secure and scalable relational data storage.
- REST APIs: For seamless communication between system components.
- Git & VS Code: For version control and efficient development.

3. Soft Skills Developed

- Teamwork and Collaboration: Participated in daily standups, sprint planning, and code reviews. Learned to integrate my work seamlessly with that of senior developers.

- **Communication Skills:** Improved both verbal and written communication through technical discussions, API documentation writing, and presenting progress to the external guides.
- **Time Management:** Managed multiple development tasks within sprint deadlines by organizing a personal task board and adhering to weekly milestones.
- **Problem Solving:** Developed the ability to research, debug, and resolve frontend and backend issues independently, using documentation, Stack Overflow, and peer collaboration.

CHAPTER 7

CHALLENGES FACED

The internship was not just a period of learning but also a journey through real-world complexities that demanded innovation, persistence, and problem-solving. Each challenge encountered was an opportunity to experiment, adapt, and grow. This chapter outlines the major hurdles and the strategies employed to overcome them.

1. Understanding the OBE Domain

Scenario: The OBE framework—including the concepts of Course Outcomes, Program Outcomes, CO-PO mapping, and attainment calculations—was entirely new at the start of the internship. Translating this educational domain knowledge into a software model required significant upfront learning.

Solution: Studied NBA accreditation guidelines and OBE documentation provided by the team. Conducted regular discussions with the external guides to clarify domain rules before translating them into database schemas and business logic.

2. Technical: Debugging, API integration, data imbalance.

The development process involved navigating complex integration and data-handling hurdles, which were resolved through systematic debugging and architectural improvements.

- **API Data Mismatches (422 Errors):** Resolved backend rejection errors by creating a frontend mapping layer to translate string-based curriculum names into the required numeric IDs.
- **Data Structure Standardization:** Addressed inconsistent nested JSON responses by implementing nullish coalescing (`?? []`) within the API client to prevent UI crashes.
- **TypeScript & Compilation Errors:** Eliminated "implicit any" and null-pointer exceptions by defining robust Interfaces and Type Guards for complex student datasets.

- **CORS & Auth Persistence:** Fixed blocked communication and session loss by configuring CORS middleware on FastAPI and using Axios Interceptors for automated JWT injection.
- **Database Integrity (Parent-Child Records):** Solved foreign key constraint failures by restructuring payloads to follow the backend's transaction pattern for session and student data.
- **Component Lifecycle Debugging:** Optimized rendering and fixed "memory leak" warnings using Browser DevTools to trace hook dependencies and effect cleanups.

3. Frontend-Backend Integration

Scenario: Integrating the FastAPI backend with the React frontend revealed several CORS errors, data format mismatches, and asynchronous state management issues during early integration testing.

Solution: Configured CORSMiddleware in FastAPI to allow requests from the React development server. Standardized API response shapes using Pydantic response models, and coordinated with the frontend team on request/response contracts before finalizing endpoint designs.

4. . Learning Curve & Tool Adaptation

- **Modern Framework Shift:** Transitioning from class components to React 18 Functional Components required mastering hooks like useEffect, useCallback, and useMemo for performance.
- **Utility-First CSS:** Moving from traditional CSS to Tailwind CSS required a mindset shift.
- **Resolution:** Extensively used official documentation and built a reusable component library to maintain design consistency.
- **Complex Data Grids:** Configuring AG Grid for large datasets (1,000+ students) initially caused UI lag.

- Resolution: Implemented Virtual Scrolling and Pagination to ensure smooth rendering and efficient memory usage.

5. Time management across tasks/modules

The internship was structured into three distinct phases, spanning twelve weeks, moving from initial requirement analysis to final deployment and documentation.

Month 1: Requirement Analysis & Core Development

- Week 1: Conducted a comprehensive system study and requirement gathering. Identified core modules and designed the initial relational database schema.
- Week 2: Developed the Announcement module and Topic Import/Management system. Created the base React UI structure with navigation and sidebars.
- Week 3: Focused on the Timetable Management backend. Developed the class scheduling module and integrated the frontend scheduling interface.
- Week 4: Performed the first phase of integration between the timetable and scheduling modules. Conducted initial system testing and resolved early-stage bugs.

Month 2: Functional Module Development

- Week 5: Built the Assignment Management and Material Sharing modules. Integrated real-time send/receive functionality for the Announcement module.
- Week 6: Developed the Quiz Management interface and enhanced the timetable with a dynamic calendar view. Implemented lesson scheduling logic.
- Week 7: Added advanced timetable utilities, including Export to PDF/Excel, and administrative options like copy, reset, and delete.
- Week 8: Executed broad system integration testing. Implemented global error handling via Axios Interceptors and refined performance for large data loads.

Month 3: Testing, Documentation & Deployment

- Week 9: Initiated full system testing. Conducted functional verification to ensure all modules (Attendance, Timetable, etc.) interacted correctly.
- Week 10: Facilitated User Acceptance Testing (UAT). Refined the UI based on feedback and fixed high-priority bugs.
- Week 11: Focused on formal documentation. Drafted technical reports, system validation documents, and summarized the project's business logic.
- Week 12: Final review and deployment of the Smart Edu-Sync platform. Prepared the final project submission and conducted mock viva sessions.

6. How the challenges were resolved.

To address the various challenges encountered during the project, a structured and professional resolution strategy was adopted. This ensured that fixes were not just temporary "patches" but sustainable improvements to the system architecture.

1. Systematic Debugging & Analysis

- Root Cause Analysis: Utilized Browser DevTools, console logging, and network tab inspection to isolate whether issues originated from frontend state logic, API connectivity, or backend data formatting.
- Incremental Fixes: Followed an iterative approach, applying small, localized code changes and verifying them through hot reloading before proceeding to larger structural updates.

2. Resource Utilization & Knowledge Sharing

- Technical Documentation: Relied heavily on the official documentation for React 18, FastAPI, and TypeScript to ensure implementation followed industry best practices.
- Collaborative Problem Solving: Engaged in Pair Programming for complex modules and conducted peer code reviews to identify edge cases and optimize logic flow.

3. Testing & Quality Assurance

- **Multi-Level Testing:** Combined Unit Testing for individual components with Integration Testing for the end-to-end data flow between the React UI and MySQL database.
- **Manual Validation:** Performed rigorous User Testing to simulate real-world faculty and student interactions, identifying UI/UX friction points.

4. Automation & Tooling

- **Static Analysis:** Enforced code quality catching syntax errors and formatting inconsistencies automatically.
- **Compile-Time Safety:** Leveraged TypeScript's compiler to identify type mismatches and potential null-pointer exceptions during development rather than at runtime.

5. Continuous Improvement

- **Milestone Retrospectives:** Held brief review sessions after completing major modules (like Attendance or Timetables) to refine development processes and improve efficiency for the next phase.

CHAPTER 8

CONCLUSION

A Journey from Concept to Implementation

The internship project, **Design and Development of course learning and attendance Management System**, was more than a technical assignment—it was a transformative journey that bridged academic knowledge with industry-level software development practice. Starting from understanding the OBE domain to designing APIs and computing attainment metrics, each phase contributed significantly to my growth as a frontend developer and future technologist.

Technical Growth through Real-World Implementation

Working with Python, Fast API, SQL Alchemy, Alembic, and JWT authentication on a live project provided deep technical exposure that classroom learning alone cannot offer. I transitioned from writing simple Python scripts to building structured, production-grade API services with proper authentication, validation, error handling, and documentation. The experience of designing a normalized database schema for a complex domain like OBE was particularly enriching.

Team Collaboration and Agile Workflow

One of the most enriching aspects of this internship was working within a collaborative development team. From sprint planning and daily standups to pull requests and code reviews—teamwork taught me the importance of clear communication, shared ownership of code, and constructive feedback. Exposure to professional tools like GitHub, Postman, and Swagger helped me understand real agile development workflows and team dynamics.

From Learner to Practitioner

This internship transitioned my role from a theoretical learner to a problem-solving practitioner. I learned to ask not just how to implement a feature, but also why a particular

design pattern or architectural decision is the right choice. Facing real challenges—such as attainment computation logic and JWT security—and delivering working solutions gave me confidence and clarity in applying software engineering principles effectively.

Alignment with Career Goals

This internship directly supports my long-term career aspirations of becoming a frontend Developer or Full-Stack Software Engineer. It validated my interest in building robust, data-driven systems that solve real institutional problems. The hands-on experience with frontend UI design, API development, database design, authentication, and integration testing has laid a strong foundation for entering industry roles with confidence. I am now better equipped to contribute meaningfully to software projects that blend technical rigour with real-world impact.

CHAPTER 9

APPENDIX

9.1 Internship Certificate



May 6, 2026

To Whom It May Concern

This is to certify that **Ms. Sarvamangala** (URN: 01FE23BCA205), a student from K.L.E Technological University, was engaged with Ionidea Interactive Pvt. Ltd. as an Intern from **03-Feb-2026 to 05-May-2026**. During her internship, she actively contributed to the "Course Learning and Assessment Management application" project, demonstrating her ability to apply theoretical knowledge in practical settings.

Throughout her tenure, Ms. Sarvamangala consistently exhibited self-motivation, diligence, and a strong sense of responsibility. She took initiative in completing tasks, collaborated effectively with team members, and displayed a keen aptitude for problem-solving.

We commend her performance during the internship and extend our best wishes for her continued success in all future endeavours.

For Ionidea Interactive Pvt. Ltd.,

Sangeeta Tandon
Director-Human Resources

Ionidea Interactive Pvt. Ltd.

CIN: U92132KA2000PTC026397

Regd. Office: 38-40 Export Promotion Industrial Park, Whitefield, Bangalore-560 066, INDIA

☎ : 91-80-66581500 ✉ : hr.bgr@ionidea.com 🌐 : www.ionidea.com

Key API Endpoints Summary

The following table summarizes the major API endpoints developed during the internship:

- POST /auth/register – Register a new user
- POST /auth/login – Authenticate and receive JWT token
- GET /courses/ – Retrieve all courses
- POST /courses/ – Create a new course
- GET /outcomes/co/{course_id} – Get COs for a course
- POST /outcomes/co-po-mapping – Map CO to PO with correlation level
- POST /marks/entry – Submit student marks for a component
- GET /attainment/{course_id} – Compute and retrieve CO attainment
- GET /reports/{programme_id} – Generate PO attainment report

9.2 Screenshots of training:

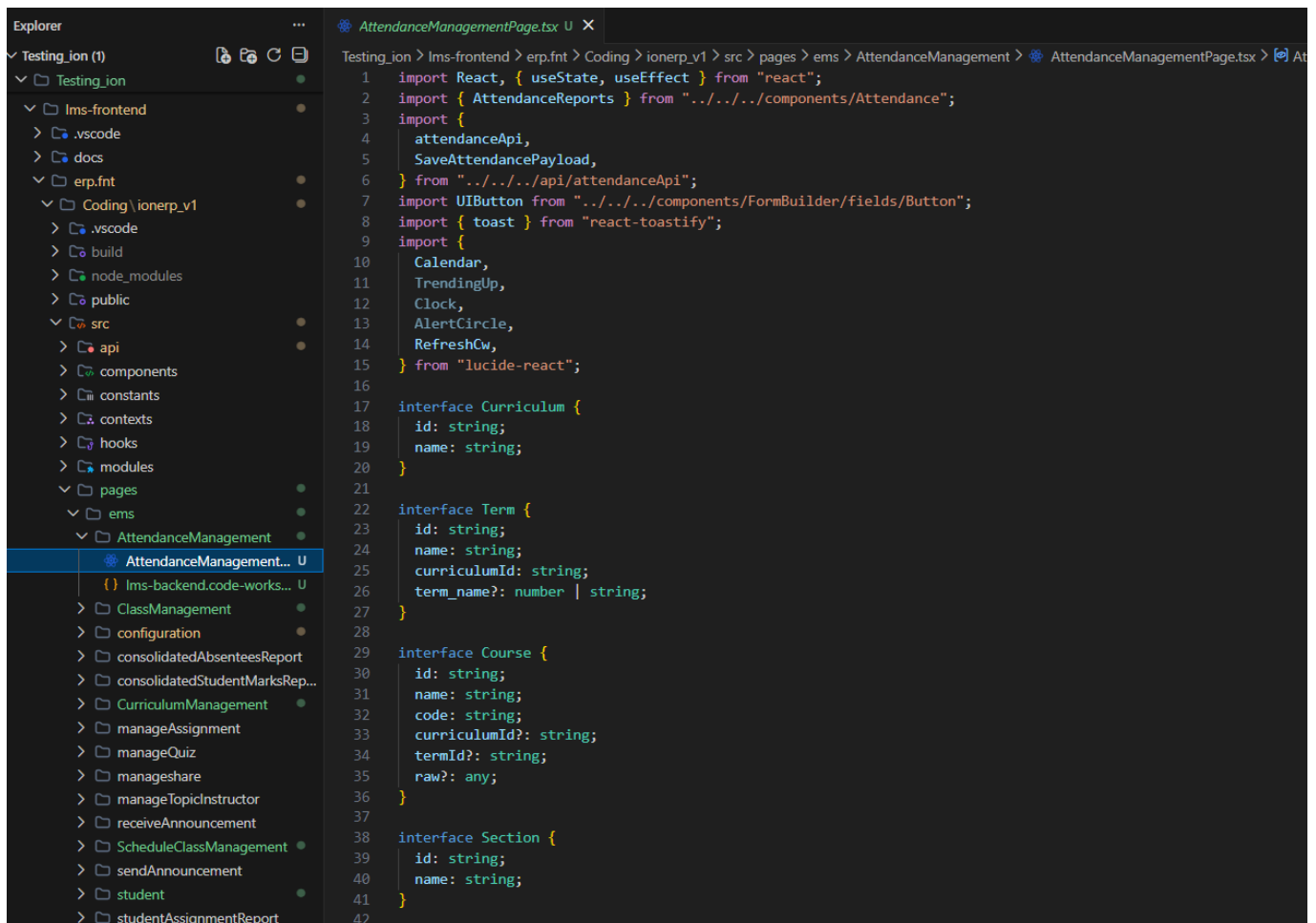


Fig 9.1 AttendanceManagementPage

It displays the AttendanceManagementPage.tsx React component, showcasing TypeScript interfaces for Curriculum, Term, Course, and Section data structures that define the core data models for the attendance management system's educational hierarchy.

```

42
43 interface Student {
44   id: string;
45   name: string;
46   rollNumber: string;
47   email: string;
48   usn?: string;
49   regno?: string;
50 }
51
52 type AttendanceStatus =
53   | "present"
54   | "absent"
55   | "late"
56   | "excused"
57   | "not marked";
58
59 interface SessionOption {
60   id: string;
61   label: string;
62   startTime: string;
63   endTime: string;
64 }
65
66 interface AttendanceRow extends Student {
67   status: AttendanceStatus;
68   remark: string;
69 }
70
71 const AttendanceManagementPage: React.FC = () => {
72   const [curriculums, setCurriculums] = useState<Curriculum[]>([]);
73   const [terms, setTerms] = useState<Term[]>([]);
74   const [courses, setCourses] = useState<Course[]>([]);
75   const [sections, setSections] = useState<Section[]>([]);
76   const [selectedCurriculum, setSelectedCurriculum] = useState("");
77   const [selectedTerm, setSelectedTerm] = useState("");
78   const [selectedCourse, setSelectedCourse] = useState<Course | null>(null);
79   const [selectedSectionId, setSelectedSectionId] = useState("");
80   const [selectedSessionId, setSelectedSessionId] = useState("");
81   const [selectedDate, setSelectedDate] = useState(
82     new Date().toISOString().slice(0, 10),
83   );
84   const [students, setStudents] = useState<Student[]>([]);

```

Fig 9.2 AttendanceManagementPage

It illustrates TypeScript interface definitions for Student, AttendanceStatus, SessionOption, and AttendanceRow that establish the data structure foundation for attendance tracking, alongside React useState hooks that manage dynamic state variables for curricula, terms, courses, sections, and attendance records.


```

71  const AttendanceManagementPage: React.FC = () => {
417      disabled={!selectedTerm || sections.length === 0}
418      >
419      <option value="">Select Section</option>
420      {sections.map((s) => (
421        <option key={s.id} value={s.id}>
422          {s.name}
423        </option>
424      ))}
425      </select>
426
427      <input
428        type="date"
429        value={selectedDate}
430        onChange={(e) => setSelectedDate(e.target.value)}
431      />
432    </div>
433
434    <div className="flex gap-2 mb-4">
435      <UIButton onClick={markAllPresent}>Mark All Present</UIButton>
436      <UIButton onClick={saveDraft}>Save Draft</UIButton>
437      <UIButton onClick={finalizeAttendance}>Finalize</UIButton>
438      <UIButton onClick={loadAttendanceRecords}>
439        <RefreshCw className="w-4 h-4" />
440      </UIButton>
441    </div>
442
443    <input
444      placeholder="Search"
445      className="mb-4 border p-2 w-full"
446      onChange={(e) => setSearchTerm(e.target.value)}
447    />
448
449    <table className="w-full">
450      <thead>
451        <tr>
452          <th>USN</th>
453          <th>Name</th>
454          <th>Status</th>
455          <th>Remark</th>
456        </tr>
457      </thead>
458      <tbody>

```

Fig 9.3 AttendanceManagementPage

The React component showcases user interface controls for attendance management, featuring dropdown selectors for section and date input, action buttons for bulk operations (Mark All Present, Save Draft, Finalize, Refresh), and a search field with table structure for displaying attendance records.

ATTENDANCE MANAGEMENT

Manage student presence, session timing, and academic progress.

[How it Works](#)

TOTAL CLASSES

12 Classes

[Quick Actions](#)

Manage Student Attendance

Campus *

AIAT-2024-2029

Term *

1 - Semester

Course *

CS101 - Computer Networks

Section *

A

Date & Session *

16/05/2024

Import Attendance

Go

Mark All Present

Mark All Absent

Search:

Search by student name or ID...

Fig 9.3 Attendance Page Output

The Attendance Management interface provides a comprehensive dashboard for faculty to filter students by curriculum, term, and course to record daily presence efficiently. It features real-time status toggles (Present/Late/Absent) and automated summary tracking to streamline institutional record-keeping and academic progress monitoring.

9.3 REFERENCES

1. Meta. (n.d.). **React – A JavaScript library for building user interfaces**. Retrieved from <https://reactjs.org/>
2. Microsoft. (n.d.). **TypeScript Documentation**. Retrieved from <https://www.typescriptlang.org/docs/>
3. Tailwind Labs. (n.d.). **Tailwind CSS Documentation**. Retrieved from <https://tailwindcss.com/docs>
4. Lucide. (n.d.). **Lucide React – Beautiful & consistent icon toolkit made for React**. Retrieved from <https://lucide.dev/>
5. React Router. (n.d.). **React Router Documentation**. Retrieved from <https://reactrouter.com/>
6. Axios. (n.d.). **Axios Documentation**. Retrieved from <https://axios-http.com/docs/intro>

7. Fkhadra. (n.d.). **React Toastify – React Notification Component**. Retrieved from <https://fkhadra.github.io/react-toastify/>
8. TanStack Query. (n.d.). **React Query Documentation**. Retrieved from <https://tanstack.com/query/latest>
9. Mozilla Developer Network. (n.d.). **Web Accessibility Initiative (WAI) - ARIA Authoring Practices**. Retrieved from <https://www.w3.org/WAI/ARIA/apg/>
10. Webpack. (n.d.). **Webpack Documentation**. Retrieved from <https://webpack.js.org/>
11. Vercel. (n.d.). **Next.js Documentation**. Retrieved from <https://nextjs.org/docs>
12. Facebook. (n.d.). **React Documentation – Concurrent Features and Suspense**. Retrieved from <https://react.dev/learn/render-and-commit>
13. Google. (n.d.). **Web.dev – Performance**. Retrieved from <https://web.dev/performance/>
14. OpenJS Foundation. (n.d.). **Node.js Documentation**. Retrieved from <https://nodejs.org/en/docs>
15. Git. (n.d.). **Git Documentation**. Retrieved from <https://git-scm.com/doc>